

Implement job sequencing with deadlines in greedy method

```
#include <stdio.h>
#include <stdlib.h>

// Structure to represent a job
struct Job {
    char id;    // Job ID
    int deadline; // Deadline of job
    int profit;  // Profit if the job is done before the deadline
};

// Comparison function to sort jobs by profit in decreasing order
int compare(const void* a, const void* b) {
    struct Job* jobA = (struct Job*)a;
    struct Job* jobB = (struct Job*)b;
    return jobB->profit - jobA->profit; // Sort in decreasing order of profit
}

// Function to find the maximum deadline among all jobs
int findMaxDeadline(struct Job jobs[], int n) {
    int max = jobs[0].deadline;
    for (int i = 1; i < n; i++) {
        if (jobs[i].deadline > max) {
            max = jobs[i].deadline;
        }
    }
    return max;
}

// Function to perform job sequencing with deadlines using the Greedy method
void jobSequencing(struct Job jobs[], int n) {
    // Sort jobs by decreasing order of profit
    qsort(jobs, n, sizeof(struct Job), compare);

    int maxDeadline = findMaxDeadline(jobs, n);

    // Array to store the result of job sequence (job id)
    char result[maxDeadline];
    // Array to keep track of free time slots
    int slot[maxDeadline];

    // Initialize all slots to be free
    for (int i = 0; i < maxDeadline; i++) {
        slot[i] = 0; // Mark all slots as empty
    }

    // Initialize result with '-' (no job assigned)
    for (int i = 0; i < maxDeadline; i++) {
        result[i] = '-';
    }

    int totalProfit = 0;
```

```

// Iterate through all jobs
for (int i = 0; i < n; i++) {
    // Find a free slot for this job, starting from its deadline
    for (int j = jobs[i].deadline - 1; j >= 0; j--) {
        if (slot[j] == 0) { // If the slot is free
            slot[j] = 1; // Mark slot as occupied
            result[j] = jobs[i].id; // Assign job ID to result
            totalProfit += jobs[i].profit; // Add profit to total
            break;
        }
    }
}

// Print the scheduled jobs
printf("Scheduled Jobs: ");
for (int i = 0; i < maxDeadline; i++) {
    if (result[i] != '-') {
        printf("%c ", result[i]);
    }
}
printf("\nTotal Profit: %d\n", totalProfit);
}

int main() {
    int n;

    // Input number of jobs
    printf("Enter the number of jobs: ");
    scanf("%d", &n);

    struct Job jobs[n];

    // Input job details (ID, deadline, profit) dynamically
    for (int i = 0; i < n; i++) {
        printf("Enter details for job %d:\n", i + 1);
        printf("Job ID: ");
        getchar(); // To consume newline character from previous input
        scanf("%c", &jobs[i].id);
        printf("Job Deadline: ");
        scanf("%d", &jobs[i].deadline);
        printf("Job Profit: ");
        scanf("%d", &jobs[i].profit);
    }

    // Display the jobs entered by the user
    printf("\nJobs Entered:\n");
    printf("ID  Deadline  Profit\n");
    for (int i = 0; i < n; i++) {
        printf(" %c    %d    %d\n", jobs[i].id, jobs[i].deadline, jobs[i].profit);
    }

    // Perform job sequencing
    printf("\nJob Sequencing with Deadlines (Greedy Method):\n");
    jobSequencing(jobs, n);

    return 0;
}

```